MASTER BUILD PROMPT

AI Sales & Outreach Agent

Full-Stack Portfolio Project — Next.js 14 + Supabase + Vercel + Claude AI

For use with Claude Code (Google Project IDX / Any Environment)

PROJECT OVERVIEW

Project Name	AI Sales & Outreach Agent
Tech Stack	Next.js 14 (App Router) + TypeScript + Supabase + Vercel
AI Engine	Anthropic Claude claude-sonnet-4-20250514 via REST API
Auth	Supabase Auth — email/password + magic link
Database	Supabase PostgreSQL — saves all prospects & generated sequences
Hosting	Vercel (auto-deploy from GitHub)
UI Framework	Tailwind CSS + Framer Motion
Accent Color	Electric Cyan #06B6D4 — sharp, sales-confident energy
Fonts	Syne (headings) + JetBrains Mono (labels/code)
Purpose	Portfolio showcase — agentic multi-asset sales workflow

WHAT YOU ARE BUILDING

A production-grade AI agent that takes a prospect's name, company, your offer, and a tone preference — then autonomously generates a complete 5-asset outreach sequence. This demonstrates a real sales automation workflow that agencies and founders would pay for.

The 5 Outreach Assets Generated

- Cold Email — subject line, personalized opener, value pitch, soft CTA, PS line (under 150 words)

- LinkedIn DM — 5 sentences max, insight-led opener, no clichés, soft ask
- Follow-Up Email — under 80 words, new angle, one-question CTA, reference to prior email
- Cold Call Script — 6 sections: OPENER, PERMISSION ASK, VALUE PITCH, DISCOVERY QUESTION, OBJECTION HANDLER, CLOSE
- Break-Up Email — 60 words, reverse psychology, removes pressure, plants future seed

> *Each asset runs as a sequential API call to Claude. The UI shows a live animated pipeline with 5 nodes lighting up in real-time — one per asset — making the agentic nature visually clear and impressive.*

THE MASTER PROMPT FOR CLAUDE CODE

Copy and paste EVERYTHING from below this line into Claude Code as your very first message in a fresh session. Do not split it across multiple messages. Do not modify it.

```
—— PASTE BELOW THIS LINE INTO CLAUDE CODE ——
```

You are an expert full-stack developer. Build me a complete, production-ready, portfolio-grade web application called the 'AI Sales & Outreach Agent'. This is a showcase project for my AI agency portfolio — it must be flawless, visually stunning, and fully functional end-to-end. Follow every instruction below precisely. Fix all errors before considering any task complete.

TECH STACK — USE EXACTLY THIS

- Framework: Next.js 14 with App Router and TypeScript strict mode
- Styling: Tailwind CSS with custom config + Framer Motion for all animations
- AI: Anthropic SDK (@anthropic-ai/sdk) — model: claude-sonnet-4-20250514
- Backend/DB: Supabase (supabase-js v2 + @supabase/ssr) — auth + PostgreSQL
- Deployment: Vercel — zero build errors, zero TypeScript warnings
- Fonts: Google Fonts — 'Syne' for headings, 'JetBrains Mono' for labels and monospace elements
- Icons: lucide-react for all UI icons

ENVIRONMENT VARIABLES

Create both .env.local (real values) and .env.example (placeholder values):

```
NEXT_PUBLIC_SUPABASE_URL=your_supabase_project_url
NEXT_PUBLIC_SUPABASE_ANON_KEY=your_supabase_anon_key
ANTHROPIC_API_KEY=your_anthropic_api_key
```

> *ANTHROPIC_API_KEY must ONLY be used server-side in API Route Handlers. Never expose it in any client component, 'use client' file, or NEXT_PUBLIC_ variable.*

SUPABASE DATABASE SCHEMA

Create supabase/schema.sql with this exact SQL. Instruct the user to run it in Supabase Dashboard > SQL Editor:

```sql
create extension if not exists "uuid-ossp";

-- Prospects table
create table public.prospects (
  id uuid default uuid_generate_v4() primary key,
  user_id uuid references auth.users(id) on delete cascade,
  prospect_name text not null,
  company text not null,
  offer text not null,
  tone text not null default 'Professional',
  created_at timestamptz default now()
);

-- Outreach sequences table
create table public.outreach_sequences (
  id uuid default uuid_generate_v4() primary key,
  prospect_id uuid references public.prospects(id) on delete cascade,
  asset_type text not null,
  -- 'cold_email' | 'linkedin_dm' | 'follow_up' | 'call_script' | 'breakup_email'
  content text not null,
  created_at timestamptz default now()
);

-- Row Level Security
alter table public.prospects enable row level security;
alter table public.outreach_sequences enable row level security;

create policy "Users own their prospects" on public.prospects
  for all using (auth.uid() = user_id);

create policy "Users own their sequences" on public.outreach_sequences
  for all using (
    prospect_id in (select id from public.prospects where user_id = auth.uid())
  );
```

EXACT FILE STRUCTURE

```
sales-agent/
  app/
    (auth)/
      login/page.tsx
      signup/page.tsx
      layout.tsx
    (dashboard)/
      dashboard/page.tsx        <- Core agent interface
      prospects/page.tsx        <- Saved prospects list
      prospects/[id]/page.tsx   <- Single prospect + sequence view
      layout.tsx                <- Protected layout with sidebar
    api/
      outreach/route.ts         <- Server-side Claude API calls
    layout.tsx                  <- Root layout with fonts + providers
    page.tsx                    <- Landing page
    globals.css
  components/
    ui/
      Button.tsx
      Input.tsx
      Badge.tsx
    layout/
      Navbar.tsx
      Sidebar.tsx
    agent/
      ProspectForm.tsx          <- The input form
      OutreachPipeline.tsx      <- Animated 5-step pipeline
      SequenceCard.tsx          <- Individual output card
      SequenceGrid.tsx          <- All 5 cards grid
    ProspectsList.tsx
    EmptyState.tsx
  lib/
    supabase/
      client.ts               <- createBrowserClient
      server.ts               <- createServerClient
      middleware.ts
    anthropic.ts              <- Anthropic client + all 5 prompts
    types.ts                  <- All TypeScript interfaces
    utils.ts                  <- Helper functions
  middleware.ts               <- Auth middleware at root
  supabase/schema.sql
  tailwind.config.ts
  next.config.ts
  .env.local
  .env.example
  README.md
```

PAGE & COMPONENT SPECIFICATIONS

1. Landing Page (app/page.tsx)

Make this the most visually impressive page — portfolio viewers land here first. Build it with:

- Full-screen hero on #080808 background with a subtle animated grid (CSS: repeating-linear-gradient lines at 1px, opacity 0.04)
- Headline: 'Your Next Customer Is One Message Away' in Syne 800 weight
- Sub-headline: 'AI-generated cold emails, LinkedIn DMs, follow-ups, call scripts, and break-up emails — for any prospect, in seconds.'
- Animated typewriter cycling through: 'Cold Email', 'LinkedIn DM', 'Follow-Up', 'Call Script', 'Break-Up Email'
- Mock UI preview: a static screenshot-style card showing the 5 pipeline nodes lit up in cyan
- 3-step 'How It Works': Enter Prospect → AI Generates 5 Assets → Copy & Send
- Social proof strip: '5 outreach assets per prospect' | 'Zero writer's block' | 'Sequences that convert'
- CTA section: 'Start for Free' (→ /signup) + 'See a Demo' (smooth scroll to demo section)

> *DESIGN DIRECTION: Dark precision. Think Stripe meets a Y Combinator-backed B2B SaaS. #080808 base, electric cyan #06B6D4 as the sole accent. Syne 800 for all display text. JetBrains Mono for all labels, tags, and stats. Sharp edges, no border-radius above 6px anywhere. Micro-borders at 1px in #1a1a1a. The feel is 'tool built by engineers, designed by someone with taste'.*

2. Auth Pages

- Minimal centered card layout on #060606 background
- Email + password fields using the custom Input component
- Magic link option via supabase.auth.signInWithOtp()
- Cyan glow on input focus: box-shadow 0 0 0 2px #06B6D433
- Redirect to /dashboard after successful auth using router.push()
- Handle both login errors and signup confirmation states inline

3. Dashboard (app/(dashboard)/dashboard/page.tsx)

This is the core agent interface. Two states:

INPUT STATE:

- ProspectForm component: two inputs side-by-side (Prospect Name, Company), full-width textarea (Your Offer), horizontal tone selector (5 options: Professional / Conversational / Confident / Friendly / Direct)
- 'Load Sample' ghost button pre-fills: prospect='Sarah Chen', company='Acme SaaS', offer='AI-powered sales automation that cuts outreach time by 70%', tone='Conversational'
- 'Generate Full Sequence' primary button in cyan — disabled when any field empty

- Below form: 5 platform tags showing what will be generated (Cold Email, LinkedIn DM, Follow-Up, Call Script, Break-Up Email)

RUNNING STATE:

- Form becomes read-only
- OutreachPipeline component: 5 circular nodes connected by horizontal lines
- Node colors: Cold Email=#06B6D4, LinkedIn=#0A66C2, Follow-Up=#F59E0B, Call Script=#10B981, Break-Up=#EF4444
- Active node: pulsing glow ring (CSS: box-shadow 0 0 20px {color}66 + scale animation)
- Done node: filled with color, shows ✓ checkmark in white
- Connecting lines fill with color as each task completes (left to right)

RESULTS STATE:

- SequenceGrid: 5 SequenceCard components in a vertical stack (not grid) for maximum readability
- Each card: collapsed by default showing first 2 lines, expand/collapse toggle, Copy button with clipboard feedback
- Cards animate in with staggered Framer Motion fadeUp (0.08s delay between each)
- 'Save to Prospects' button saves the prospect + all 5 outputs to Supabase
- 'Run New Sequence' button resets entire form and state

4. Prospects Page (app/(dashboard)/prospects/page.tsx)

- List of all saved prospects from Supabase for the logged-in user
- Each row: prospect name, company, date, 5 cyan asset badges, 'View' button
- Click a prospect row → navigates to /prospects/[id]
- Beautiful empty state: icon + 'No prospects saved yet' + 'Run your first sequence →' CTA
- Sort by: Most Recent (default), Name, Company

5. Single Prospect View (app/(dashboard)/prospects/[id]/page.tsx)

- Shows prospect details at the top (name, company, offer, tone, date)
- All 5 generated assets displayed as expanded SequenceCards
- 'Copy All' button that copies all 5 assets as formatted text
- 'Delete Prospect' button (with confirmation) — permanently removes from Supabase

6. API Route (app/api/outreach/route.ts)

- POST handler only
- Validate request body: { assetType: string, prospectName: string, company: string, offer: string, tone: string }
- Verify Supabase auth via createServerClient — return 401 if not authenticated

- Use Anthropic SDK with ANTHROPIC_API_KEY from process.env (server-only)
- Call the correct prompt from lib/anthropic.ts based on assetType
- Return { result: string } on success
- Return { error: string } with appropriate status codes on failure

THE 5 AGENT PROMPTS — COPY THESE EXACTLY INTO lib/anthropic.ts

> *These prompts are engineered for output quality. Do not paraphrase or modify them. Copy them character-for-character.*

```
export type AssetType = 'cold_email' | 'linkedin_dm' | 'follow_up' | 'call_script'
| 'breakup_email';


interface ProspectInput {
  prospectName: string;
  company: string;
  offer: string;
  tone: string;
}


export const OUTREACH_PROMPTS: Record<AssetType, (p: ProspectInput) => string> = {


  cold_email: ({ prospectName, company, offer, tone }) => `
You are an elite B2B sales copywriter. Write a cold email to ${prospectName} at
${company}.
Offer/Value prop: ${offer}
Tone: ${tone}
Rules:
- Subject line that earns an open (no clickbait, no fake 'Re:')
- Personalized opener that references something real about their role or company
- 2-3 short paragraphs, never more
- Clear value prop in one sentence
- Soft CTA (ask for 15 mins, not a sale)
- PS line that adds social proof or urgency
- Total body: under 150 words
Output format — SUBJECT: then BODY: then PS: — nothing else.`,


  linkedin_dm: ({ prospectName, company, offer, tone }) => `
You are a LinkedIn outreach expert with a 40%+ reply rate.
Write a LinkedIn DM to ${prospectName} at ${company}.
Offer/Value prop: ${offer}
Tone: ${tone}
Rules:
- Never start with 'I hope this message finds you well'
- Lead with a genuine observation about them, their company, or their industry
- Maximum 5 sentences total
- End with a soft, low-commitment ask
- No emojis unless tone is Friendly
```

```
Output ONLY the DM text, nothing else.`,

  follow_up: ({ prospectName, company, offer, tone }) => `
You are a B2B sales expert. Write a follow-up email to ${prospectName} at
${company}
who has not replied to a cold email about: ${offer}
Tone: ${tone}
Rules:
- Reference the previous email without being needy or aggressive
- Add a new angle, insight, or data point — not just 'bumping this up'
- Under 80 words in the body
- End with exactly one direct question
Output format — SUBJECT: then BODY: — nothing else.`,

  call_script: ({ prospectName, company, offer, tone }) => `
You are a senior sales trainer. Write a cold call script for reaching
${prospectName} at ${company}.
Offer/Value prop: ${offer}
Tone: ${tone}
Include these exact sections with these exact labels:
[OPENER] - 5-second pattern interrupt, state your name and company
[PERMISSION ASK] - one sentence asking if they have 27 seconds
[VALUE PITCH] - the 15-second elevator pitch, outcome-focused
[DISCOVERY QUESTION] - one open question that uncovers pain
[OBJECTION: NOT INTERESTED] - 2 sentences, acknowledge + reframe
[CLOSE] - book a specific time, give two options
Output ONLY the script with section labels, nothing else.`,

  breakup_email: ({ prospectName, company, offer, tone }) => `
You are a sales psychologist. Write a break-up email to ${prospectName} at
${company}
after multiple unanswered outreaches about: ${offer}
Tone: ${tone}
Rules:
- Completely remove all sales pressure
- Be respectful and human
- Plant a seed for them to reach out when timing is right
- Under 60 words in the body
- Witty without being snarky — this sometimes re-engages cold prospects
Output format — SUBJECT: then BODY: — nothing else.`
};
```

TAILWIND CONFIG & DESIGN SYSTEM

tailwind.config.ts — add these extensions:

```
extend: {
  colors: {
    brand: {
      DEFAULT: '#06B6D4',
      light:   '#67E8F9',
```

```
      dark:    '#0E7490',
      glow:    '#06B6D433',
    },
    surface: {
      1:      '#080808',
      2:      '#0c0c0c',
      3:      '#111111',
      border: '#1a1a1a',
    },
  },
  fontFamily: {
    display: ['Syne', 'sans-serif'],
    mono:    ['JetBrains Mono', 'monospace'],
  },
  animation: {
    'glow-pulse': 'glowPulse 2s ease-in-out infinite',
    'fade-up':    'fadeUp 0.4s ease forwards',
  },
  keyframes: {
    glowPulse: { '0%,100%': { opacity: 1 }, '50%': { opacity: 0.4 } },
    fadeUp: { from: { opacity: 0, transform: 'translateY(12px)' }, to: { opacity:
1, transform: 'translateY(0)' } },
  },
}
```

ANIMATIONS — FRAMER MOTION SPECS

- Page entry: staggerChildren 0.08s, each child: opacity 0→1, y 16→0, duration 0.35s ease
- Pipeline node active: scale 1→1.08 with 2s loop, glow box-shadow pulse
- Pipeline node done: background fills with color, checkmark scales in (0→1, spring)
- Connecting line: width 0→100% as left node completes, 0.3s ease
- SequenceCard open: height auto with AnimatePresence, 0.4s ease
- Copy button success: color change to #10B981, icon swap, 2s then revert
- Landing hero: stagger reveal each word of headline, 0.04s delay per word

SIDEBAR NAVIGATION (dashboard layout)

- Fixed left sidebar: 240px wide on desktop, collapsible on mobile
- Logo: 'SalesAgent' in Syne bold with cyan dot as accent
- Nav items: Dashboard (home icon), Prospects (users icon), Settings (gear icon)
- Active item: left border 2px solid #06B6D4, background #06B6D410
- Bottom of sidebar: user email + Logout button

VERCEL DEPLOYMENT

- Create vercel.json: { "framework": "nextjs" }
- Ensure 'npm run build' passes with zero errors locally before deploying
- Add all env vars to Vercel: Settings > Environment Variables
- Configure Supabase Auth redirect URLs to include your Vercel domain: https://your-app.vercel.app/**
- Add Vercel domain to Supabase: Dashboard > Authentication > URL Configuration

ERROR PREVENTION — READ AND IMPLEMENT PROACTIVELY

> *These are the most common errors. Fix them before they happen — do not wait for runtime failures.*

Supabase SSR Auth (Most Common Failure)

- Use @supabase/ssr — NOT @supabase/auth-helpers-nextjs (deprecated)
- createBrowserClient in client components, createServerClient in server components + route handlers
- middleware.ts at the project root must refresh the session cookie on every request
- Never call supabase.auth.getUser() in client components to protect routes — use middleware

```
// middleware.ts
import { createServerClient } from '@supabase/ssr'
import { NextResponse } from 'next/server'
import type { NextRequest } from 'next/server'

export async function middleware(request: NextRequest) {
  let response = NextResponse.next({ request: { headers: request.headers } })
  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    { cookies: { getAll() { return request.cookies.getAll() },
                 setAll(cs) { cs.forEach(({ name, value, options }) => {
                   request.cookies.set(name, value)
                   response.cookies.set(name, value, options) }) } } }
  )
  const { data: { user } } = await supabase.auth.getUser()
  if (!user && request.nextUrl.pathname.startsWith('/dashboard')) {
    return NextResponse.redirect(new URL('/login', request.url))
  }
  return response
}
export const config = { matcher: ['/dashboard/:path*', '/prospects/:path*'] }
```

Anthropic API — Server-Side Only

- Import Anthropic ONLY in files that are server-side: api routes, server components, server actions
- The api/outreach/route.ts file must have NO 'use client' directive — it is server-only by default
- Always wrap Anthropic calls in try/catch and return typed error responses
- For network timeouts: set AbortController with 30s timeout on the fetch

TypeScript — Zero 'any' Policy

- Define ProspectRow and OutreachSequenceRow interfaces matching your Supabase tables exactly
- Type all useState hooks explicitly: useState<string>(''), useState<Record<AssetType, string>>({})
- Type all API route params and return values
- Run tsc --noEmit before considering any file complete

Common Next.js 14 App Router Mistakes

- 'use client' components cannot be async — move async logic to server components or hooks
- Do not use useRouter from 'next/navigation' in server components
- loading.tsx files prevent the page from being empty during data fetching — add one per route
- error.tsx files must be 'use client' components
- Dynamic routes ([id]) need generateStaticParams or dynamic = 'force-dynamic' export

README.md CONTENT

Write a comprehensive README.md that includes:

- Project title with cyan accent badge: '□ AI Sales & Outreach Agent'
- One-liner description + live demo URL placeholder
- Features list with emoji bullets for each of the 5 assets
- Tech stack table (framework, AI, database, hosting)
- Local setup: clone → npm install → .env.local → Supabase schema → npm run dev
- Supabase setup: create project, run schema.sql, configure auth redirect URLs
- Vercel deployment: push to GitHub → import to Vercel → add env vars → deploy
- Screenshots section (placeholder)
- License: MIT

FINAL VERIFICATION CHECKLIST

> *Run through every item below before calling the project done. Claude Code must verify each one.*

- npm run build — zero errors, zero TypeScript errors
- All 5 agent assets generate correct, high-quality output from Anthropic API
- Auth: sign up, log in, log out, session persists on page refresh
- Saving to Supabase works — prospects and sequences appear in the DB
- Prospects page loads saved data correctly per user
- Single prospect view shows all 5 assets
- Mobile responsive: 375px, 768px, 1440px — all layouts correct
- Dark theme consistent — no white flash on navigation
- Copy button works and shows visual feedback
- Empty states exist for prospects page
- Error states exist for API failures
- Middleware protects /dashboard and /prospects routes
- Vercel deploy completes successfully from GitHub push

— END OF CLAUDE CODE PROMPT —

HOW TO USE THIS DOCUMENT

1. Open idx.dev → New Workspace → Blank → Next.js
2. Open the AI/Claude Code assistant panel in IDX
3. Copy the full prompt from Section 3 and paste it as your first message
4. Create a Supabase project at supabase.com → run the SQL schema from Section 3
5. Copy your Supabase URL + anon key into .env.local
6. Get an Anthropic API key at console.anthropic.com → add to .env.local
7. Push project to GitHub (IDX has built-in Git)
8. Import GitHub repo into Vercel → add all 3 env vars → deploy
9. Add your Vercel URL to Supabase Auth redirect URLs → test end-to-end
10. Add the live URL to your portfolio!

> *If Claude Code gets stuck on a specific section, paste just that section again and say: 'Focus only on this section. Complete it fully with no errors before moving on.'*